Question 4: Docker Network - Demonstrate communication between two containers through a Docker network, using container names and container IP addresses.

Course outcome: CO1, CO2

A user-defined bridge network was created with two containers. They communicated by container name (embedded DNS) and by IP address. The default bridge network was used as a contrast: it has no embedded DNS.

Network and containers

```
$ docker network create lab2net
$ docker run -d --name net-a --network lab2net ubuntu:24.04 sleep infinity
$ docker run -d --name net-b --network lab2net ubuntu:24.04 sleep infinity

$ docker inspect net-a --format "{{.NetworkSettings.Networks.lab2net.IPAddress}}"
net-a IP: 172.18.0.2
$ docker inspect net-b --format "{{.NetworkSettings.Networks.lab2net.IPAddress}}"
net-b IP: 172.18.0.3
```

Communication by container name (embedded DNS)

```
$ docker exec net-a ping -c 2 net-b
PING net-b (172.18.0.3) 56(84) bytes of data.
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=1 ttl=64 time=9.50 ms
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=2 ttl=64 time=0.090 ms
2 packets transmitted, 2 received, 0% packet loss
```

Communication by container IP address

```
$ docker exec net-a ping -c 2 172.18.0.3
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.116 ms
2 packets transmitted, 2 received, 0% packet loss
```

Contrast: default bridge network has no embedded DNS

```
$ docker run -d --name net-c ubuntu:24.04 sleep infinity
$ docker exec net-c ping -c 1 net-a
ping: net-a: Name or service not known
--> user-defined networks are required for name resolution
```

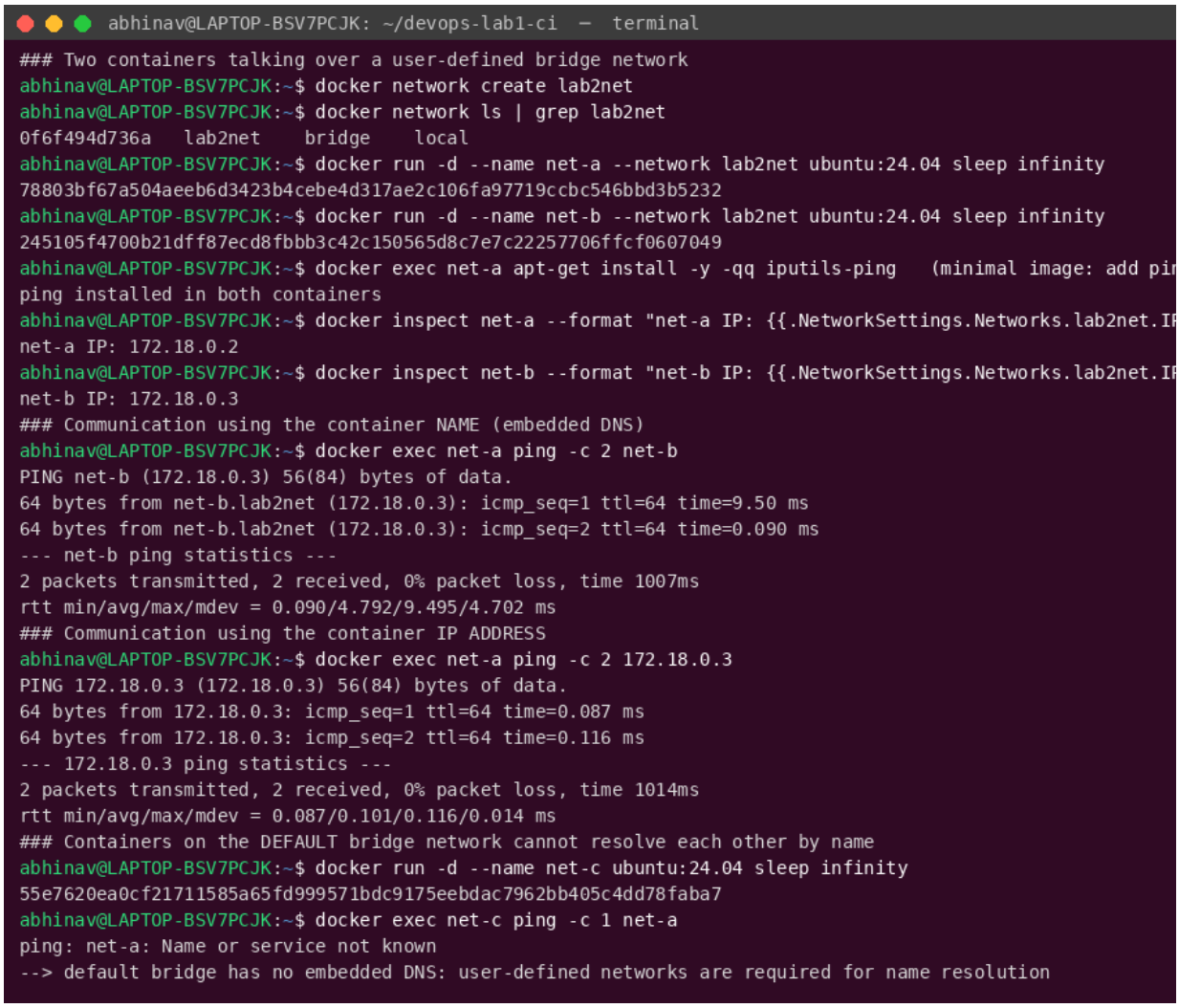


Fig: Full network demo: name resolution, IP ping and the default-bridge contrast

Observations

- User-defined bridge networks provide automatic DNS between containers; the default bridge does not.
- IP-based communication works on any network but IPs change when containers are recreated - names are stable.
- The minimal ubuntu:24.04 image ships without ping; iputils-ping was installed inside the demo containers.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Docker, Docker Compose, Minikube and kubectl. The evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-2 under the docs/ directory.